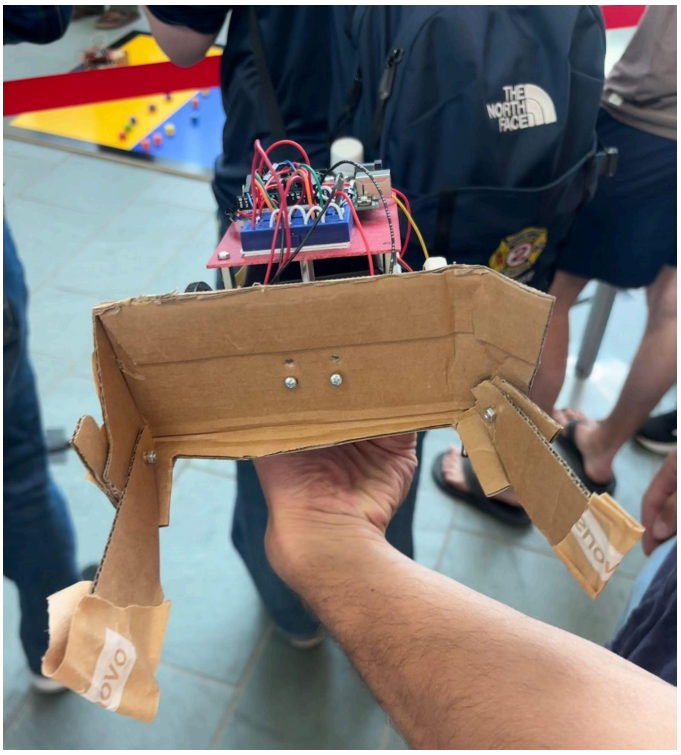
Milo Rappoport MDR255, Parker Ashworth PA367, Thomas Meyer TM676
Team 56
5/13/2026

Mechatronics Robot Project Final Report

1. Robot Design and Strategy Overview



The design for our robot included two servo motors for the wheels, a color sensor attached to the bottom and a mechanical arm-like drop-down plow mechanism. Our robot had the plow at the front, then the color sensor right behind the plow on the bottom front of the chassis, two wheels on either front side of the chassis and the sphere on the back of the chassis. The six-volt battery pack is attached beneath the breadboard, on top of the chassis. It is held in place with standoffs and rubber bands spanning horizontally the side of the robot. This allows ease of access to the batteries which we would need to exchange frequently. We also attached the nine-volt battery which powers the arduino uno board below, fastened with tape. The mechanical plow was made out of cardboard. For this component, we used cardboard because it was free and readily available as well as easy to cut into shape. We also considered how the card might have some elasticity if we came into slight contact with another robot compared to more stiff materials like plastic filament or acrylic. It can still be caught and take our robot off course but in the case where the robots only have a slight contact, cardboard might bend where plastic would put us farther off of our desired line. We attached the plow with screws and designed it to fit exactly within the constraints of the 8x8 inch box to maximize block gathering area. The two arms were connected loosely enough to balance upright at the beginning of the match but immediately fell down once the robot moved. The extended arms are also angled outward as shown in the picture above to further increase block gathering area. Fully extended, the robot would fit perfectly inside the 12 inch diameter circle constraint. Our strategy was to keep it simple, drive quickly to the middle line on the right side and turn down the middle line and drive until we hit the other side. We discovered that our robot would turn to the left as it drove forward. We thought that the motors were slightly different, but after testing several different servos, we concluded that this was not the issue. We tried various other mitigation ideas but, in the end, we opted to keep the “drifting” effect because it let us block the other robot a bit as we collected the middle blocks by driving onto their side and then coming

back to ours. To execute our strategy we aimed the robot at the right, middle of the board, knowing it would drift left slightly as it drove forward. We coded our robot to first have a small forward-and-backward to ensure the arms dropped, then drive the measured distance to the middle line, then turn 90 degrees left, then drive until we hit black. We also added a rubber band on the left wheel to increase traction on the left wheel and attempt to mitigate the slight left drift of our robot. While not completely negating the drift, we found that this did noticeably decrease the amount the robot strayed from driving straight forward. We positioned the arduino board toward the back with the bread board in front to allow for more easy wiring of the motors as well as color sensor. This also allowed the power wiring for starting and stopping the match to be easily accessible.

2. Design Process Reflection

In our design pitch, we brainstormed a lot of different strategies but ultimately concluded that the best way to win the competition was to collect at least half the blocks as quickly as possible and then sit with them in a safe place until the round was over. We wanted to move quickly before the other robot reached the middle and could interfere with our gathering. We decided that the most effective way to do that was to hard code driving straight at the middle of the board, then turn one way and drive until we hit the black board, and then stop. We would detect the border using our color sensor. This would allow us to also potentially use the color sensor to detect the middle border where yellow meets blue. In the end, we opted not to use the color sensor to do this because we felt it would overcomplicate our algorithm. During the second milestone, we discovered that our robot drifts left significantly due to some imbalance between the right and left side drivetrains. After various testing including adding a rubber band to the left wheel the drift became small enough that we could complete milestone two by just changing the time spent turning left versus right. When we were doing milestone three we learned that the lighting on the board had a significant impact on the color sensor because we would have a working algorithm on one board, and then switch to a more shadowy board and the algorithm would stop working. This convinced us to hard-code our initial drive to the center line instead of relying on the color sensor to drive a set distance, because it removed unnecessary complexity that could cause our robot to fail on competition day. Milestone three also reinforced our drive straight at the middle strategy, because it was much slower to interact with the border as milestone three forced us to correct when at the black border as opposed to stopping in place as we desired for our final competition strategy. During milestone four, we considered several different block collection ideas, such as intakes, one-way gates, and sweepers, but ultimately decided on a cardboard extendable plow because it could fit a lot of blocks, was easy to repair on competition day and allowed our drop down mechanical arms that held in place with just friction. This would save us the trouble of mounting, wiring and programming two extra servos which we felt would be unnecessary. When we were testing for milestone four, we were regularly getting about half of the blocks, so we decided it would be more efficient to start at one side of the middle line and attempt to get all of the blocks. This way we did not risk getting fewer than half of the blocks or the other team’s robot getting some of the blocks from our side. The code we used for milestone four was the same code that we used for the competition, which aligns with

our final strategy of getting as many blocks as we can and then sitting with them off the board away from the other team's robot.

3. Competition Analysis

   The first two matches of the competition went well. Our robot deployed its extenders, moved toward the middle right of the board, and then turned to sweep up some cubes. Those first two wins highlighted the major strength of our robot which was its simplicity. Many other robots had complex methods to eventually collect the majority of the cubes on the board, but when facing those types of robots, ours collected a couple cubes and displaced the rest making it much more difficult for the opponent to collect cubes in the way they intended. However, the third match demonstrated the biggest flaw in our strategy. As soon as the match started, the opponent's robot swept up the majority of the cubes very quickly and displaced the rest leaving our comparatively slow robot nothing left in its fixed path. If we had faced any more robots with similar strategies as that one, it most likely would have ended in the same way because ours was not very fast and had to head toward the side of the board before it could ever pick up any cubes. In the other three matches the robot would drive to the side and turn as planned, but it would not move forward. We think the lighting in the competition area was darker than in the lab which caused the robot to think it was on the black section instead of the blue part. Based on the opponent's robots in those matches, it is hard to say who would have won if our robot functioned as intended, but it did not, so naturally we lost those three matches.

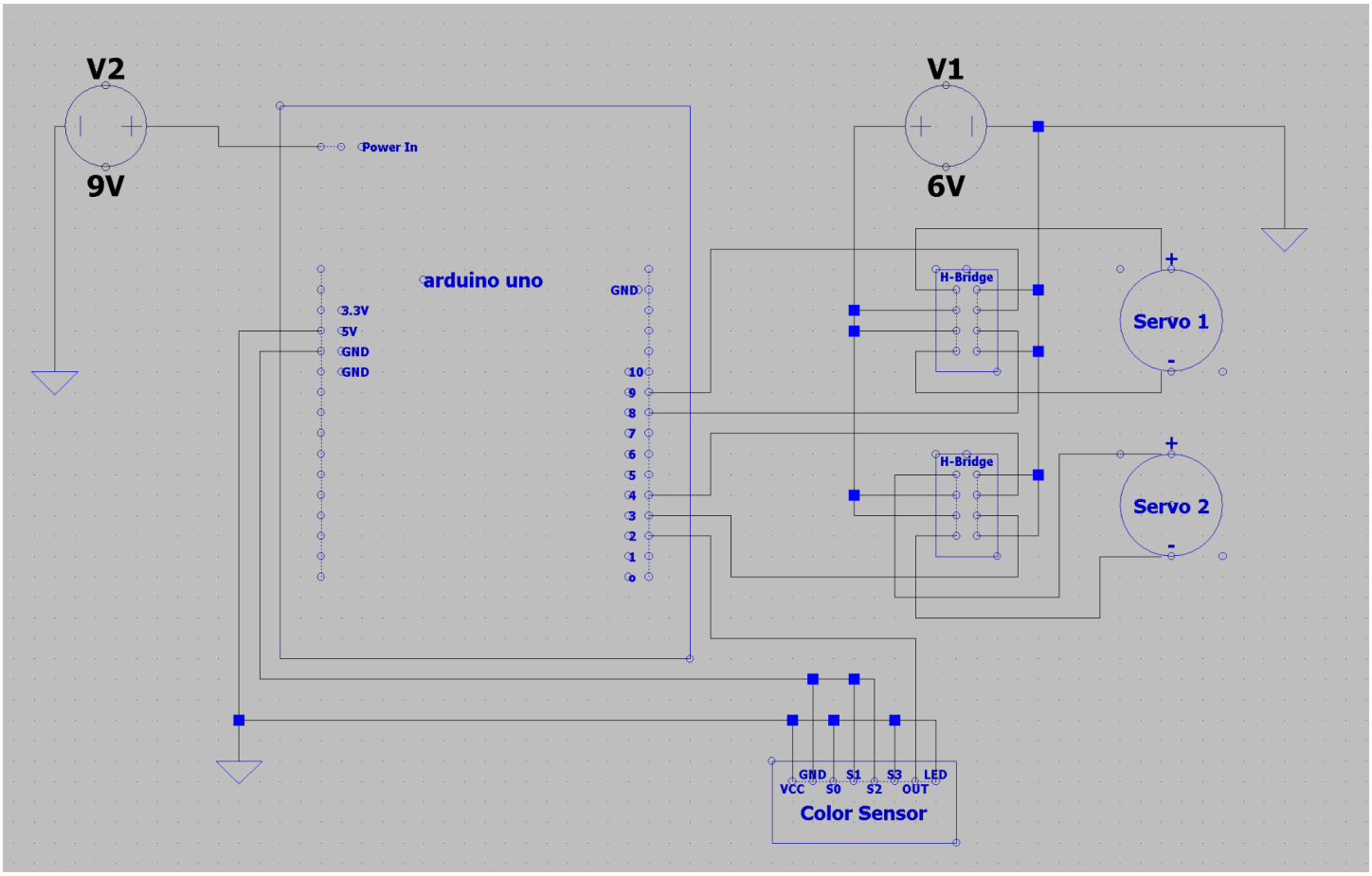
4. Conclusions

   Having completed the project once, there are a couple key takeaways that would help us on another go around. First, make sure to test the robot in the competition environment. If we had done this, we may have realized that the lighting change does affect the color sensor and we could have helped the robot differentiate between blue and black better. We knew this might be an issue from milestone three, but if we had tested in different lighting we could have performed much better at competition. We would need to change the upper bound of the blue to be higher, possibly from 350 to 400. Next, you should also have a plan for when you or the cubes get knocked out of their intended position. We found that simple for the most part was better, but if the other robot messes up your plan, you should have a way to deal with that. This could also be simple, but something is better than nothing. Finally, if done correctly, fast is the way to go. It would be harder to pull off and require more fine tuning, but if your robot can collect a lot of cubes extremely quickly, there is almost no opposing strategy that can stop you. If we were doing it over again, it would probably be worth finding wheels that increase our robots speed, and we would make sure to test the robot thoroughly in competition lighting. Our advice to next year's students is that simple is better and speed is key. A lot of the robots we faced that had fancy algorithms were slower to get blocks at the beginning and spent the rest of the match driving around not collecting anything.

5. Appendix A: Bill of Materials

Item Number	Name	Quantity	Weight	Price
1	Repurposed cardboard Box	1	44g	~2$ new
2	H-Bridge	1	25g	3$
3	QTI sensor	1	2g	2$
4	Rubber Band	3	<1g	~0.06$
5				
6				
7				
				TOTAL: 7.18$

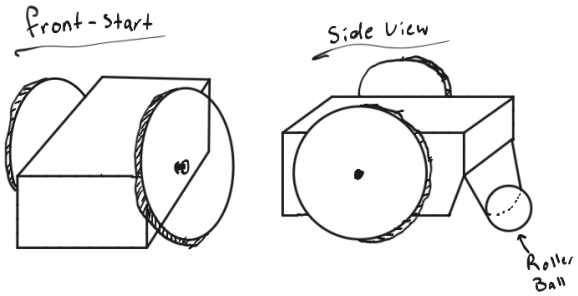
6. Appendix B: Circuit Diagram

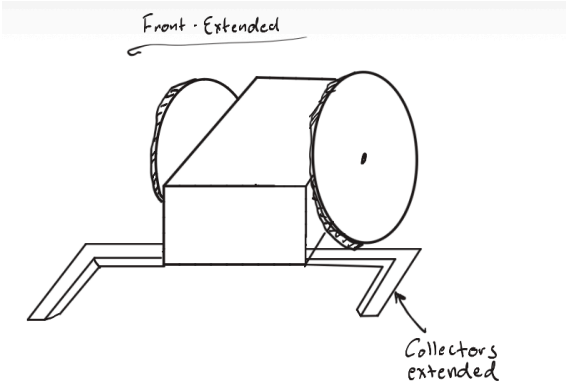


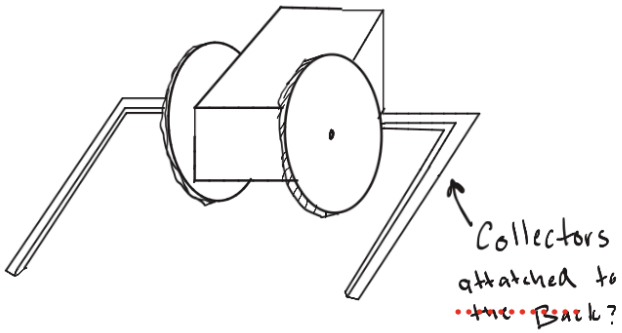
*The above circuit diagram (generated using LTspice) shows the electronic circuit of our robot. The 9V battery powers the arduino board and plugs into the power port. The 6V battery pack which powers the servos consists of four 1.5V AA batteries. It connects to each of the two H-bridges which are used for motor control. Everything has common ground as shown in the diagram. The motor control pins on the Arduino board are pins 3 and 4 as well as pins 8 and 9. The pin to read the color sensor output was pin 2. The color sensor has a combination of 5V and GND connections corresponding to detecting black similar to Lab 4.

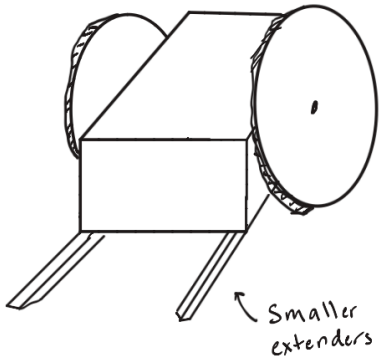
7. Appendix C: Design Drawings

Brainstorming:

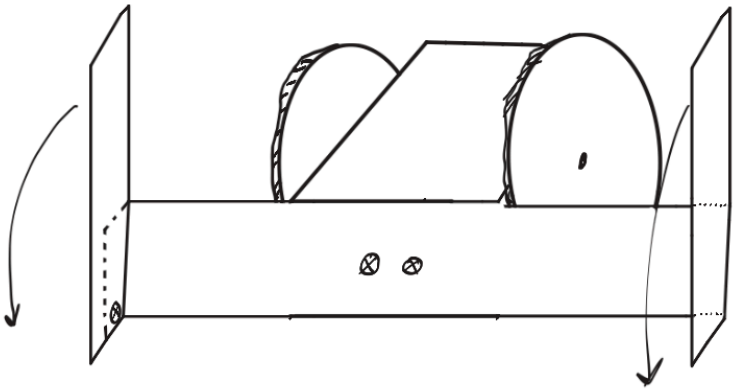








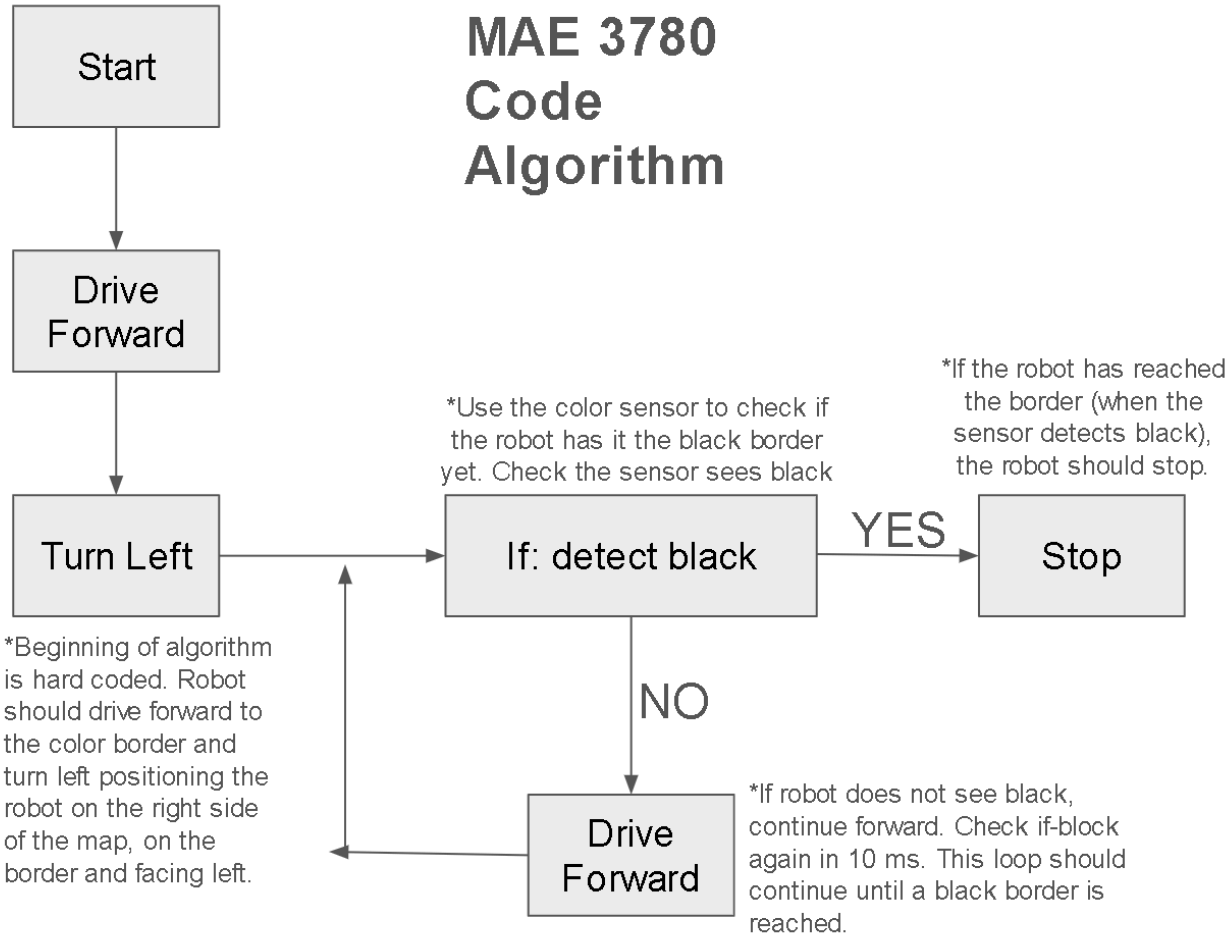
Final Design:



Cardboard Plate
$20\,cm \times 7cm \times 0.4cm = 56\,cm^3$
Arms
$2 \times (10cm \times 4cm \times 0.4cm) = 32\,cm^3$
$\rho_{cardboard} \approx 0.5\,g/cm^3$
$W_{cardboard} = 44g$

*The arms (shown left) rotate down to extend the arms to increase the block gathering area within the robot's boundary. There is no servo powered mechanism required as the arms simply fall into place upon driving forward. We found that a slight perturbation causes the arms to fall so we simply implemented a segment where we drive forward and backward quickly at the start of the match to ensure the arms drop into place.

8. Appendix D: Flowchart of Code



9. Appendix E: Final Arduino Code

```
#include <avr/io.h>
#include <avr/interrupt.h> //Needed for hardware interrupts
#include <util/delay.h> //Needed for delays
#include <stdlib.h> //Needed for abs()

//Global Variables
volatile int period = 0;
volatile unsigned int timer = 0;
//Interrupt function: ISR - resets and reads the timer value
ISR(PCINT2_vect){
  if (PIND & (1 << 2)) {
    TCNT1 = 0; //Reset the timer to zero on a rising edge (PD2 high)
  }
  else{
    timer = TCNT1; //store the timer value on a falling edge
  }
}

//Initializes the interrupt and timer
void initColor(){

  DDRD &= ~(1 << 2); //initialize pin 2 input for color sensor

  //Initialize pin change interrupt
  PCICR |= (1 << 2);          //Enable PCIE2
  PCMSK2 &= ~(1 << 2);        //Keep disabled until needed

  //Initialize timer 1
  TCCR1A = 0x00;              //Normal mode
  TCCR1B = (1 << CS10);       //Prescaler 1
  TCNT1 = 0;                  //Initialize

  sei();                      //Enable global interrupts
}

//Calculates the period of the sensor output wave
int getColor(){
```

```
  //Enable PCINT18 (PD2)
  PCMSK2 |= (1 << 2);

  //Add delay to allow an interrupt to trigger
  _delay_ms(10);

  //Disable PCINT18 to prevent further interrupts
  PCMSK2 &= ~(1 << 2);

  //Return the period
  period = timer / 8;
  return period;
}

//drive forward for certain time (ms)
void drive_forward_delay(int t){
  PORTB |= (1 << 1);  PORTB &= ~(1 << 0); //give power to right servo (forward)
  PORTD |= (1 << 3);  PORTD &= ~(1 << 4); //give power to left servo (forward)

  // Loop 't' times, delaying 1ms each loop
  for (int i = 0; i < t; i++) {
    _delay_ms(1);
  }
}

//drive backward for certain time (ms)
void drive_back_delay(int t){
  PORTB |= (1 << 0);  PORTB &= ~(1 << 1); //give power to right servo
(backward)
  PORTD |= (1 << 4);  PORTD &= ~(1 << 3); //give power to left servo (backward)

  // Loop 't' times, delaying 1ms each loop
  for (int i = 0; i < t; i++) {
    _delay_ms(1);
  }
}

//Turn right for certain time (ms) -- right wheel backward, left wheel forward
```

```
void drive_left_delay(int t){
  PORTB |= (1 << 0);  PORTB &= ~(1 << 1); //give power to right servo (forward)
  PORTD |= (1 << 3);  PORTD &= ~(1 << 4); //give power to left servo (backward)

  // Loop 't' times, delaying 1ms each loop
  for (int i = 0; i < t; i++) {
    _delay_ms(1);
  }
}

//Stop both motors for given time
void stop(int t){
  PORTB &= ~((1 << 0) | (1 << 1)); // Right motor stop
  PORTD &= ~((1 << 3) | (1 << 4)); // Left motor stop

  // Loop 't' times, delaying 1ms each loop
  for (int i = 0; i < t; i++) {
    _delay_ms(1);
  }
}

//Main function: start of match: action starts here
int main(void){
  int currentColor; //Store value of color from sensor
  DDRB |= (1 << 0) | (1 << 1); //pins 8 & 9 as outputs for servo motor
  DDRD |= (1 << 3) | (1 << 4); //pins 3 & 4 as outputs for servo motor

  Serial.begin(9600); //Communication protocol for arduino board (baud rate =
9600 bit/s)
  initColor(); //run init function to initialize Interrupt and Timer

  // Give the sensor a moment to stabilize on startup
  _delay_ms(100); //code waits for 0.1s

  while(1){
    currentColor = getColor(); //store color value in variable
    //Serial.println(currentColor); //print out this value for debugging
purpose
```

```
  //Drive forward and backward  quickly to deploy arm extenders
  drive_forward_delay(100); //drive forward for 0.1s
  drive_back_delay(100);    //drive backward for 0.1s
  drive_forward_delay(100); //drive forward for 0.1s
  drive_back_delay(100);    //drive backward for 0.1s

  //Drive forward to the middle
  drive_forward_delay(3000); //drive forward for 3s
  drive_left_delay(600); //Turn left for 0.6s

  //Loop to drive forward until black border is detected
  while(1){
    currentColor = getColor(); //Sense color and store in variable

    //Check if color is black (value for black tested in the lab was 350)
    if (currentColor >= 350){ //
      //Our adjustment algorithm for hitting border from earlier testing
      //decided not to use this section because we kept losing blocks
      /*stop(100);
      drive_back_delay(400);
      stop(100);
      drive_right_delay(200);
      */
      stop(7000); //If color is black, stop, we have reached the border
      stop(7000);
      stop(7000);
      stop(7000);
      stop(7000);
      stop(7000);
      stop(7000);
      break; //end while loop
    }
    else {
      drive_forward_delay(10); //if color is not black, continue forward
    }
  }
}
```

```
}
```